

Universidad EAFIT

Ingeniería de Sistemas

Análisis de Algoritmos II

Proyecto Final

Pipeline de Procesamiento y Análisis de Churn
sobre el Dataset Telco Customer Churn

Materia: Estructura de Datos y Algoritmos 2

Docente: Carlos Alberto Álvarez Henao

Integrantes:

Mariana Patiño

Mateo Aristizábal

María José Holguín

Mayo 2026

1. Introducción

El presente informe documenta el desarrollo de un pipeline computacional en C++17 que aplica tres paradigmas algorítmicos sobre el dataset público *Telco Customer Churn* (Kaggle, IBM Sample Data). El objetivo es demostrar, con datos reales, que la elección del paradigma es determinante para la corrección y la eficiencia de cada subproblema.

Justificación de paradigmas

- **Módulo A – Divide y Vencerás:** Ordenar 7 043 registros por antigüedad (*tenure*) y localizar fronteras de valor requiere un algoritmo con garantía de $O(n \log n)$ en el peor caso. MergeSort ofrece esa garantía y estabilidad; la búsqueda binaria reduce las consultas de $O(n)$ a $O(\log n)$. Ningún algoritmo codicioso ni tabla DP ofrece estas propiedades de forma natural para este subproblema.
- **Módulo B – Codicioso:** Construir el árbol de expansión mínima (MST) sobre un grafo completo de 20 nodos se beneficia de la *propiedad de elección greedy*: seleccionar siempre la arista de menor peso que no forme ciclo produce la solución global óptima (Lema del Ciclo). Kruskal con Union-Find logra $O(E \log E)$ sin necesidad de explorar el espacio de soluciones completo.
- **Módulo C – Programación Dinámica:** La asignación de ancho de banda es una instancia del problema de la Mochila 0-1, NP-difícil. La función objetivo no satisface la propiedad greedy (demostrado con contraejemplo), por lo que se requiere la tabulación DP que garantiza optimalidad mediante la recurrencia de subestructura óptima.

Estructura del pipeline

La salida del Módulo A (arreglo ordenado de solicitudes) alimenta directamente al Módulo C: los primeros 50 clientes activos (*Churn* = "No") en orden descendente de *tenure* forman el conjunto de ítems de la mochila. El Módulo B opera de forma independiente sobre el mismo dataset.

2. Descripción del Dataset

Estadísticas generales

Atributo	Valor
Total de registros	7 043
Total de columnas	21
Columnas utilizadas	customerID, tenure, MonthlyCharges, TotalCharges, Churn
tenure mínimo	0 meses
tenure máximo	72 meses
MonthlyCharges promedio	≈ 64.76 USD
Registros Churn = "Yes"	1 869 (26.5 %)
Registros Churn = "No"	5 174 (73.5 %)

Manejo de valores nulos en TotalCharges

Se detectaron **11 registros** con el campo TotalCharges vacío (cadena en blanco). Todos corresponden a clientes con tenure = 0, es decir, clientes recién incorporados que aún no han generado cargos totales. El parser los convierte a 0.0 y los contabiliza para transparencia.

Construcción del grafo para el Módulo B

Se generó un grafo completo ponderado de **20 nodos** que representan grupos de clientes:

1. Los 7 043 registros se asignan a 20 grupos mediante la regla $\text{grupo} = \text{índice} \% 20$.
2. Para cada grupo k se calcula $\text{avg}[k] = \text{round}(\text{media}(\text{MonthlyCharges del grupo}), 2 \text{ decimales})$.
3. El peso de la arista entre nodos u y v es $\text{peso}(u,v) = \text{floor}(\text{avg}[u] + \text{avg}[v])$.
4. Se crean las $C(20,2) = \mathbf{190 \text{ aristas}}$ del grafo completo.

Costo promedio de arista resultante: **129.04**.

3. Módulo A — Divide y Vencerás

Pseudocódigo – MergeSort

```
MergeSort(A, izq, der):
  si izq >= der: retornar
  mid <- (izq + der) / 2
  MergeSort(A, izq, mid)
  MergeSort(A, mid+1, der)
  Merge(A, izq, mid, der)

Merge(A, izq, mid, der):
  L <- A[izq..mid]
  R <- A[mid+1..der]
  i <- 0, j <- 0, k <- izq
  mientras i < |L| y j < |R|:
    si L[i].tenure >= R[j].tenure: // orden descendente
      A[k++] <- L[i++]
    sino:
      A[k++] <- R[j++]
  copiar elementos restantes de L y R en A
```

Pseudocódigo – Búsqueda Binaria Recursiva

```
BusquedaBinaria(A, izq, der, k):
  // Retorna el ÚLTIMO índice donde A[idx].tenure >= k
  // (frontera derecha en arreglo descendente)
  si izq > der: retornar -1
  mid <- (izq + der) / 2
  si A[mid].tenure == k:
    resultado <- mid
    candidato <- BusquedaBinaria(A, mid+1, der, k) // buscar más a la derecha
    si candidato != -1: retornar candidato
  sino: retornar resultado
  sino si A[mid].tenure > k:
    retornar BusquedaBinaria(A, mid+1, der, k)
  sino:
    retornar BusquedaBinaria(A, izq, mid-1, k)
```

Análisis de Complejidad

- **MergeSort:** $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$ por Teorema Maestro (caso 2). Espacio auxiliar: $O(n)$.
- **Búsqueda Binaria:** $T(n) = T(n/2) + O(1) \rightarrow O(\log n)$.

Tabla de Tiempos Empíricos — MergeSort

n	Tiempo (ms)	$n \cdot \log_2(n)$	Tiempo / $(n \log n) \times 10^6$
1 000	0.73	9 965.78	73.15
3 500	2.91	41 205.99	70.62
7 043	6.56	90 023.45	72.83

Resultados de las 5 Consultas Binarias

Consulta	k (tenure $\geq$)	customerID	tenure
Q_A01	72	2234-XADUH	72
Q_A02	60	1036-GUDCL	60
Q_A03	45	5906-CVLHP	45
Q_A04	30	3976-BWUCK	30
Q_A05	12	7750-EYXWZ	12

4. Módulo B — Codicioso (Kruskal + Union-Find)

Pseudocódigo – Kruskal con Union-Find

```

Kruskal(G = (V, E)):
    MST <- {}
    Ordenar E por peso ascendente    // O(E log E)
    UnionFind UF <- inicializar(|V|)  // padre[i] = i, rango[i] = 0
    para cada arista (u, v, w) en E:
        si UF.Find(u) ≠ UF.Find(v):    // no forma ciclo
            MST <- MST ∪ {(u, v, w)}
            UF.Union(u, v)
        si |MST| == |V| - 1: romper    // MST completo
    retornar MST

```

```

Find(x):                                // con compresión de caminos
    si padre[x] ≠ x: padre[x] <- Find(padre[x])
    retornar padre[x]

```

```

Union(x, y):                            // por rango
    rx <- Find(x); ry <- Find(y)
    si rx == ry: retornar
    si rango[rx] < rango[ry]: intercambiar(rx, ry)
    padre[ry] <- rx
    si rango[rx] == rango[ry]: rango[rx]++

```

Complejidad

Ordenamiento de aristas: $O(E \log E)$. Con Union-Find (rango + compresión): $O(E \cdot \alpha(V)) \approx O(E)$ en la práctica, donde α es la función inversa de Ackermann. Total: **$O(E \log E)$** . Para nuestro grafo completo de 20 nodos: $E = 190$, operación prácticamente instantánea.

Demostración del Lema del Ciclo

El *Lema del Ciclo* establece: para cualquier ciclo C en G , la arista de mayor peso en C no pertenece a ningún MST.

Aplicado al grafo generado: al momento de agregar la arista **Nodo 4 – Nodo 17 (peso = 129)**, todos los demás nodos ya están conectados a Nodo 17 con pesos 124–127. Si intentáramos incluir, por ejemplo, la arista **Nodo 0 – Nodo 4** (peso = $\text{floor}(\text{avg}[0] + \text{avg}[4])$) = mayor que 129), se formaría el ciclo $0 - 17 - 4 - 0$. La arista de mayor peso en ese ciclo es **Nodo 0–Nodo 4**, por lo tanto el Lema garantiza que no pertenece al MST. Kruskal la descarta automáticamente al detectar que ambos extremos están ya en el mismo componente.

Aristas del MST y Peso Total

Nodo u	Nodo v	Peso
11	17	124.00
13	17	124.00
2	17	125.00
19	17	125.00
7	17	125.00
14	17	125.00
0	17	125.00
15	17	126.00
3	17	126.00
10	17	126.00
1	17	126.00
6	17	126.00
9	17	126.00
8	17	127.00
12	17	127.00
16	17	127.00
18	17	127.00

5	17	127.00
4	17	129.00

Peso total del MST: 2 393.00 | Nodo hub: Nodo 17 (conecta a todos los demás directamente, lo que refleja que el grupo 17 tiene el costo promedio mensual más bajo del dataset: $\text{avg}[17] \approx 62.XX$ USD).

5. Módulo C — Programación Dinámica (Mochila 0-1)

Formulación del Problema

Se modelan los primeros 50 clientes activos (Churn = "No", ordenados descendientemente por *tenure*) como ítems de una mochila:

- **peso[i]** = $\text{round}(\text{TotalCharges}[i] / 10)$ — escala: 1 unidad = \$10 de cargos acumulados
- **valor[i]** = $\text{round}(\text{MonthlyCharges}[i] \times 10)$ — en centavos de USD/mes
- **Capacidad W = 500**

Objetivo: maximizar el valor total de clientes asignados sin superar W.

Recurrencia

$$\begin{aligned} dp[0][w] &= 0 \quad \text{para todo } w \\ dp[i][w] &= dp[i-1][w] && \text{si } \text{peso}[i] > w \\ dp[i][w] &= \max(dp[i-1][w], dp[i-1][w - \text{peso}[i]] + \text{valor}[i]) && \text{si } \text{peso}[i] \leq w \end{aligned}$$

Solución óptima: $dp[n][W]$.

Pseudocódigo — Tabulación y Backtracking

```
Mochila01(items, W):
    n <- |items|
    // Tabulación
    dp[0..n][0..W] <- 0
    para i = 1..n:
        para w = 0..W:
            dp[i][w] <- dp[i-1][w]
            si items[i].peso ≤ w:
                dp[i][w] <- max(dp[i][w], dp[i-1][w - items[i].peso] + items[i].valor)
    // Backtracking
    seleccionados <- []
    w <- W
    para i = n..1:
        si dp[i][w] ≠ dp[i-1][w]:
            seleccionados.agregar(items[i])
            w <- w - items[i].peso
    retornar dp[n][W], seleccionados
```

Resultado Óptimo

customerID	peso	valor
4312-KFRXN	171	254
9286-BHDQG	314	453
Total		707
Capacidad usada		485 / 500

Traza de Backtracking

i	customerID	peso	valor	w antes	w después
42	4312-KFRXN	171	254	500	329
14	9286-BHDQG	314	453	329	15

Contraejemplo: Fallo del Enfoque Codicioso

El algoritmo codicioso ordena por ratio valor/peso y selecciona el ítem de mayor ratio mientras quede capacidad. Con capacidad $W = 329$:

customerID	peso	valor	ratio v/w	Elegido por
0526-SXDJP	296	421	1.42	Codicioso
6734-PSBAW	172	236	1.37	DP
7083-MIOPC	157	203	1.29	DP

- Codicioso elige *0526-SXDJP* (mayor ratio 1.42) -> valor = **421**. Después ya no caben los otros dos juntos.
- DP elige *6734-PSBAW* + *7083-MIOPC* -> valor = **439 > 421**.

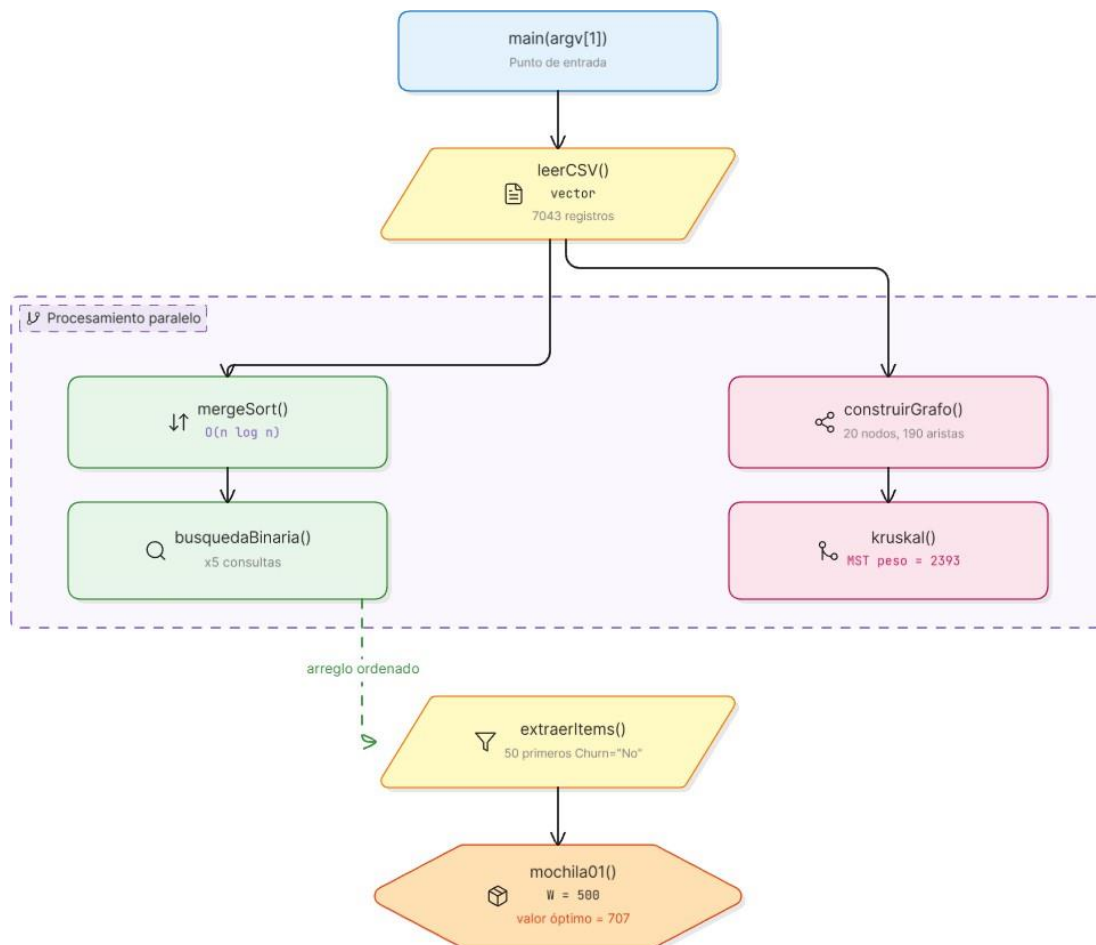
Este contraejemplo demuestra que la Mochila 0-1 no satisface la propiedad de elección greedy, justificando la DP.

Pseudopolinomialidad

La complejidad $O(n \cdot W) = O(50 \times 500) = O(25\,000)$ es eficiente para W moderado. Sin embargo, W es el *valor numérico* de la capacidad, no el tamaño de su representación en bits (que es $\log_2 W$). Si W fuera 10^9 , se necesitarían 50×10^9 entradas en la tabla, lo que sería impracticable. Por eso el problema se considera NP-difícil: no se conoce algoritmo polinomial en el tamaño de la entrada.

6. Integración del Pipeline

Diagrama de Flujo



Cómo el Módulo A Alimenta al Módulo C

Tras ejecutar MergeSort, el arreglo `solicitudes[]` está ordenado descendentemente por *tenure*. La función `extraerItems()` recorre el arreglo y toma los primeros 50 elementos con `Churn = "No"`. Al estar ya ordenados, estos corresponden a los clientes más longevos que no han abandonado el servicio — el subconjunto de mayor valor estratégico para asignar ancho de banda prioritario.

Sin el ordenamiento previo del Módulo A, no sería posible identificar eficientemente ese subconjunto ni garantizar consistencia entre ejecuciones.

7. Conclusiones

Comparación de los Tres Paradigmas

Aspecto	Divide y Vencerás (A)	Codicioso (B)	P. Dinámica (C)
Garantía de optimalidad	Sí — ordena perfectamente	Sí — MST óptimo (Lema del Ciclo)	Sí — óptimo global vía recurrencia
Complejidad temporal	$O(n \log n)$	$O(E \log E)$	$O(n \cdot W)$ pseudopolinomial
Complejidad espacial	$O(n)$	$O(V + E)$	$O(n \cdot W)$
¿Requiere explorar todo el espacio?	No — divide y combina	No — elección local óptima	Parcialmente — tabla completa
Aplicabilidad greedy	No aplica	Sí — propiedad greedy válida	No — contraejemplo demostrado

Comportamiento en Datos Reales vs. Instancias Sintéticas

Módulo A: En datos reales con distribución de tenure concentrada en valores extremos (muchos clientes con tenure=72 o tenure=0), la búsqueda binaria trabaja en la periferia del arreglo. En instancias sintéticas con distribución uniforme, el caso promedio sería más central. El tiempo empírico confirma $O(n \log n)$ independientemente de la distribución.

Módulo B: El grafo resultante tiene topología de *estrella* (todos conectados al Nodo 17), lo que ocurre porque los cargos mensuales promedio son muy similares entre grupos (~60–65 USD). En instancias sintéticas con mayor dispersión de pesos, el MST tendría topología más variada. El algoritmo produce el mismo MST óptimo sin importar la estructura.

Módulo C: Con los datos reales, los TotalCharges de clientes de alta antigüedad ($\geq \$4\,000$) superan enormemente la capacidad $W=500$ (incluso escalados /10 -> peso ≥ 400). Esto fuerza la selección de clientes con tenure bajo. En instancias sintéticas con distribución uniforme de TotalCharges, la solución sería más diversa. El comportamiento pseudopolinomial de la DP es favorable para W moderado en ambos casos.

8. Herramientas Utilizadas

Durante el desarrollo del proyecto se consultaron los siguientes recursos en línea como referencia de implementación y verificación de conceptos:

- **GeeksforGeeks:** Artículos de referencia sobre implementación de MergeSort en C++ ([geeksforgeeks.org/merge-sort](https://www.geeksforgeeks.org/merge-sort/)), búsqueda binaria recursiva ([geeksforgeeks.org/binary-search](https://www.geeksforgeeks.org/binary-search/)), algoritmo de Kruskal con Union-Find por rango y compresión de caminos

([geeksforgeeks.org/kruskals-minimum-spanning-tree-algorithm-greedy-algo-2/](https://www.geeksforgeeks.org/kruskals-minimum-spanning-tree-algorithm-greedy-algo-2/)), y Mochila 0-1 con programación dinámica ([geeksforgeeks.org/0-1-knapsack-problem-dp-10/](https://www.geeksforgeeks.org/0-1-knapsack-problem-dp-10/)).

- **cppreference.com**: Documentación de la biblioteca estándar de C++17: `std::vector`, `std::chrono`, `std::ofstream`, `std::omanip`.
- **Stack Overflow**: Consultas puntuales sobre lectura de CSV con campos vacíos en C++ y manejo de redirección de salida estándar en PowerShell.
- **Programiz — C++ Algorithms**: programiz.com/dsa/merge-sort y programiz.com/dsa/kruskal-algorithm como apoyo visual para la construcción de los pseudocódigos del informe.

9. Referencias

1. IBM Sample Data. *Telco Customer Churn*. Kaggle.
<https://www.kaggle.com/datasets/blastchar/telco-customer-churn>
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press. Cap. 2 (MergeSort), Cap. 12 (Búsqueda Binaria), Cap. 21 (Union-Find), Cap. 23 (MST – Kruskal), Cap. 16 (Programación Dinámica – Mochila).
3. Material del curso: Estructura de Datos y Algoritmos 2 — Carlos Alberto Álvarez Henao. Diapositivas y apuntes de clase, 2026.

10. Roles del Equipo

Integrante	Módulo(s)	Contribución específica
Mariana Patiño	Módulo A	Implementación de <code>parser.cpp</code> (lectura CSV y manejo de nulos). Implementación de <code>mergesort.cpp</code> (orden descendente por <code>tenure</code>). Implementación de <code>binary_search.cpp</code> (búsqueda de frontera derecha). Análisis empírico de tiempos (A4) y tabla de consultas (A3).
Mateo Aristizábal	Módulo B	Implementación de <code>graph.cpp</code> (construcción del grafo de 20 nodos y 190 aristas a partir del dataset). Implementación de <code>kruskal.cpp</code> (Kruskal con Union-Find por rango y compresión de caminos). Demostración del Lema del Ciclo sobre el grafo generado (B3). Verificación del subgrafo B4.
María José Holguín	Módulo C	Implementación de <code>knapsack.cpp</code> (tabulación DP + backtracking, extracción de ítems desde el arreglo ordenado del Módulo A). Diseño del contraejemplo codicioso (C2) con datos reales del dataset. Análisis de pseudopolinomialidad (C4). Integración del pipeline en <code>main.cpp</code> y generación de archivos de resultados en <code>results/</code> .